Flip and Find Game: A Memory Match Game in C++ with SFML

Abstract

This report presents the design and implementation of "Flip and Find," a memory-based card matching game developed in C++ using the Simple and Fast Multimedia Library (SFML). The game challenges players to flip and match card pairs as quickly as possible. The gameplay is enriched by difficulty settings, an animated graphical user interface (GUI), a timer, and a leaderboard that stores the top 15 scores. The objective of this project is to combine data structures such as vectors, priority queues, and deques with interactive UI design to produce an enjoyable and functional gaming experience. The project demonstrates fundamental programming principles and serves as an educational resource for students learning game development using C++ and SFML.

Introduction

Games are powerful learning tools in the development of programming skills. "Flip and Find" is a simple memory game where the player must find pairs of matching cards by flipping them two at a time. The game promotes cognitive engagement and challenges a player's memory and speed. Built using SFML, this game demonstrates how multimedia libraries can be used to build real-time interactive applications in C++. The project includes features like game state management, animated interactions, mouse-based input handling, timer tracking, difficulty selection, and file-based score saving and loading. It is designed with a modular architecture to improve readability, scalability, and user engagement.

Related Work

Memory matching games have been widely implemented on both mobile and desktop platforms. Classic examples include Memory, Concentration, and Matching Pairs. On the technical side, projects built with JavaFX, Python (Tkinter or Pygame), and Unity are common in introductory programming courses. However, fewer examples use C++ with SFML, making this project particularly relevant. Similar SFML-based card games usually demonstrate sprite management and animation but rarely include full leaderboard logic or GUI complexity. By using STL containers like vector, priority_queue, and deque, this implementation stands out for blending game logic with practical file I/O and real-time UI updates.

Objective

The primary objectives of this project are:
- To develop a visually appealing and functional card matching game in C++ using SFML.
- To demonstrate the use of STL containers (vector, priority_queue, deque) in a practical scenario.
- To implement difficulty levels that scale the number of cards and game complexity.
- To build a leaderboard that tracks and stores the best 15 scores with time and difficulty.
- To create a game that includes a menu, rules popup, and resume functionality with intuitive UI interactions.
- To provide a reusable and educational codebase for beginner-to-intermediate level game developers.

Main Functions

1. Game Initialization and GUI:
- SFML is used to create a RenderWindow of size 1000x800.
- Fonts are loaded, and buttons for “New Game,” “Resume,” “Difficulty,” “Leaderboard,” and “Exit” are created.

2. Card Generation (createCards):
- A vector of card IDs is shuffled randomly.
- Cards are laid out in a grid using rows and columns based on difficulty.
- Each card is an object that includes id, matched, revealed, and SFML graphical properties.

3. Difficulty Setting (updateDifficultySettings):
- Easy: 8 cards (4 pairs), 4 columns.
- Normal: 16 cards, Hard: 32 cards, Master: 64 cards.

4. Leaderboard Handling:
- Scores are stored using a ScoreEntry struct.
- saveScore() writes new scores to a file and updates a deque of max 15 entries.

5. Game Logic:
- Mouse events detect card flips and button clicks.
- Two flipped cards are compared — if matched, they’re permanently revealed.

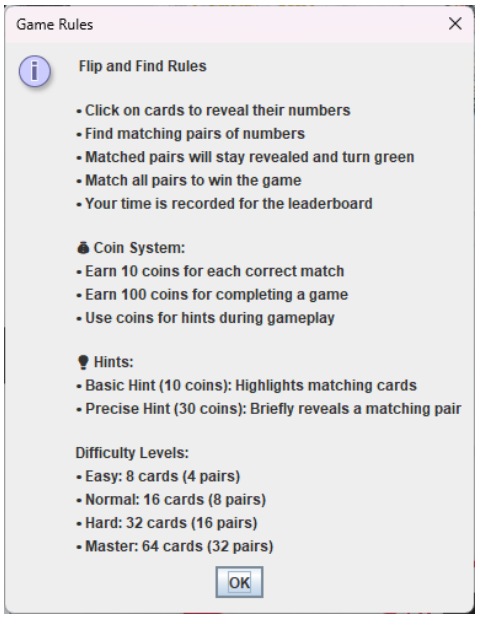
6. Graphics and Effects:
- Hover effects change card colors.
- Real-time timer shows elapsed time during the game.

Screenshots of Project

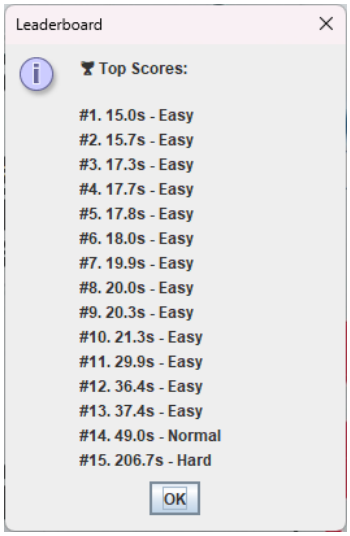
Note: Screenshots should be captured manually during gameplay.
Suggested screenshots to include:
1. Main Menu – showing buttons.

2. Game Board – showing flipped cards.

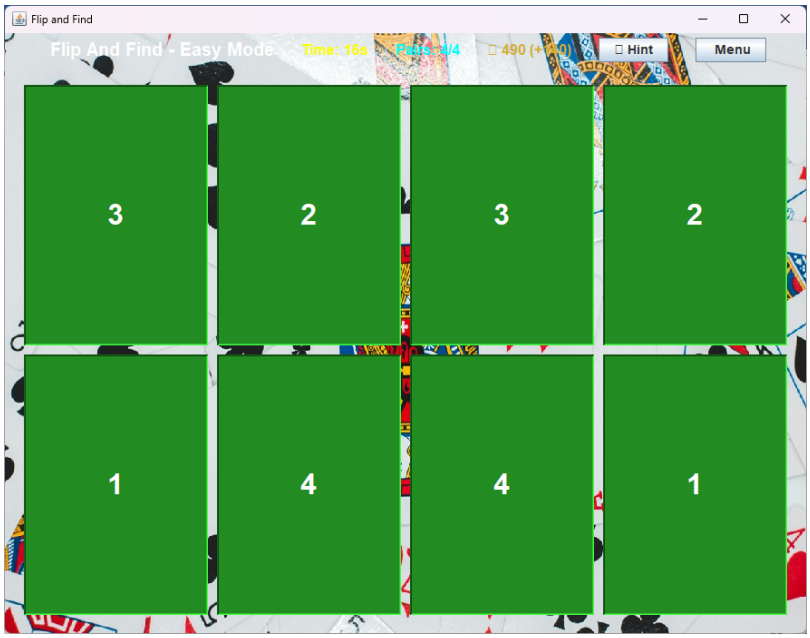
3. Rules Popup – before game starts.



4. Leaderboard Screen – after scores are added.



5. End of Game – all cards matched.



Future Work

1. Card Graphics:
- Replace numbers with images or icons.
2. Sound Effects:
- Add click and match sounds.
3. Profile and Score Saving:
- Introduce player names.
4. Game Resume Logic:
- Save and load card states.
5. Themes and Skins:
- Allow different UI skins.
6. Multiplayer Mode:
- Add local 2-player mode.
7. Mobile Version:
- Port the game to Android or Web.